https://pubsonline.informs.org/journal/isre

信息系统研究 提前发表文章，第1–20页 ISSN 1047–7047(印刷版), ISSN 1526–5536(网络版)

开源软件开发中的工作流自动化：通过机械化与编排加速创新

黄奥、黄霓、洪一力

迈阿密大学赫伯特商学院，佛罗里达州科勒尔盖布尔斯，邮编33146

通讯作者

联系方式：alanhuang@miami.edu，https://orcid.org/0009–0004–7530–1251（黄安）；nhuang@miami.edu，https://orcid.org/0000–0003–3416–513X（黄南）；khong@miami.edu，https://orcid.org/0000–0002–0577–7877（黄康）

收稿日期：2024年9月29日

修订日期：2025年6月18日；2025年10月2日；2025年11月12日

收稿日期：2025年11月22日

在线发表于预印文章：2026年1月12日

https://doi.org/10.1287/isre.2024.1551

版权信息： 2026 INFORMS

摘要。本研究构建了一个概念框架，区分工作流自动化的两种基础机制——机械化与编排，并在开源软件开发背景下实证检验工作流自动化如何加速创新。基于流程自动化、创新及开源软件开发领域的文献，我们提出：(a) 工作流自动化能加速开源软件创新；(b) 不同的工作流自动化机制（机械化与编排）分别对渐进式创新（即维护性工作）和实质性创新（即新功能开发）产生差异化影响。通过对2019至2020年间超过4500个GitHub代码库及28万个议题的纵向观测，我们采用嵌入式多层混合研究方法，将机器学习与计量经济学相结合。为验证假设，计量分析进一步运用前瞻滚动进入匹配法，并配合双重差分法和工具变量等多种估计策略。分析结果首先表明，工作流自动化（特指GitHub Actions）使议题解决速度提升10.1%，平均每个议题节省4.3天，显著提升了创新速度。其次，我们特别区分机械化与编排机制，深入探究二者如何差异化影响软件开发中的两类创新。研究发现：机械化使维护性渐进式创新速度显著提升7.8%，平均每个议题节省3.0天，而编排机制的影响不显著；相反，在新功能开发这类实质性创新中，机械化作用有限，而编排机制使议题解决速度提升16.6%，平均每个议题节省9.1天。通过反事实估计进一步分析下游效应发现，速度提升并未牺牲质量——工作流自动化显著增加了贡献量（已关闭议题数）和项目质量（星标、分叉与发布版本）。本研究揭示了工作流自动化的差异化机制：机械化加速以常规维护为导向的渐进式创新，而编排机制能催化以创造性开发为导向的实质性创新。通过论证工作流自动化不仅加速创新，更重塑了开源软件开发中创新展开的底层过程，本研究为自动化、创新及软件开发领域的研究与实践提供了重要启示。

历史：高级编辑 Gal Oestreicher–Singer；副编辑 Min–Seok Pang。

资助声明：本研究由OpenAI研究员访问计划及谷歌云研究奖提供支持。

补充材料：在线附录可在 https://doi.org/10.1287/isre.2024.1551 获取。

关键词： 工作流自动化●流程自动化●机械化●编排●创新速度●渐进式创新●实质性创新●开源软件开发(OSS)

1. 引言

开源软件（OSS）开发已成为全球软件产业的基石，推动着大规模的协作创新（Osterloh and Rota 2007, Levine and Prietula 2014）。随着数据科学和人工智能（AI）的迅猛发展，OSS是鉴于其开发透明且负责任的系统的潜力，这一点正变得日益重要(Mikalef等人，2025)。在这个技术快速发展的时代，自动化正在深刻重塑软件开发。近期许多关注点集中在像GitHub Copilot这样的生成式AI编码工具上，它们自动化

通过生成代码和促进自动完成来进行内容创作，从而影响开发者的生产力（Cui等人，2024；Song等人，2024；Yeverechyahu等人，2024）。随着自动化在各行业扩散并渗透到工作的几乎每个方面——引发了关于增强与替代的争论（Acemoglu和Restrepo，2019；Brynjolfsson，2023；Li等人，2025）——本文聚焦于一个较少被讨论但日益关键的前沿领域：自动化那些协调和维持协作的过程，通常被称为工作流自动化。我们试图理解工作流自动化的本质及其对开源软件开发创新的影响，这是一个严重依赖分布式创造力和协作解决问题的领域。

工作流自动化的起源可追溯至信息系统(IS)研究领域关于流程创新、工作流管理系统与业务流程再造的开创性研究(Davenport 1993, Hammer and Champy 1993)。工作流管理系统旨在管理与优化任务序列，通常通过整合多样化工具与人工输入以实现目标成果(Georgakopoulos et al.1995, Basu and Kumar 2002)。当代软件开发已从管理工作流转向自动化流程，典型案例如支持持续集成(CI)与持续交付/部署(CD)流水线的系统，这些系统实现了从代码提交到部署的全流程自动化(Maruping and Matook 2020)。除CI/CD外，GitHub等平台通过GitHub Actions(GHA)进一步普及工作流自动化，使开发者能够通过可配置、可复用的代码单元（即action）在其代码仓库中定义、执行并自动化定制工作流。

在现代软件开发中，保持竞争优势关键取决于创新速度(Kessler and Chakrabarti 1996)。快速的技术变革与不断变化的市场动态使得通过开发新解决方案并持续优化现有代码库来实现快速创新变得至关重要。在开源软件开发中，加快创新速度（此处定义为处理众包问题请求所需的时间）的需求更为关键，因为缓慢的错误修复或未能引入新功能可能导致社区参与度下降，并最终导致项目被弃用。开源项目具有去中心化治理、自愿贡献和通常松散耦合的协作结构等特点(Shah 2004, Howison and Crowston 2014, Safadi et al. 2021)，为研究创新速度（包括渐进式创新即维护和实质性创新即新开发）提供了丰富场景。尽管开源项目的自主性和扁平化层级等特点能促进创造力与创新(Amable 1988, Perry-Smith and Shalley 2003)，但同时也带来了显著的协调挑战(Lindberg et al.2016, Mockus et al. 2002)是有效协作工作系统(Crowston 1997)的基本前提。工作流自动化系统(例如GHA)在开源软件项目中，对加速维护导向和开发导向的创新都具有显著潜力。

本研究从概念层面提出并实证检验工作流自动化是否以及通过何种机制加速开源软件开发中的创新。我们引入了一种新的工作流自动化机制分类法：机械化，侧重于自动化执行定义明确、自包含且可重复的离散任务（如编译代码、运行测试）；以及编排化，侧重于自动化协调任务序列或不同贡献者、管理其间的依赖关系、并促进跨阶段信息流动（如发送通知、管理依赖项升级）。我们认为这些机制对维护导向型与发展导向型创新具有差异化影响。理解这一区别至关重要，因为最适合可预测、常规任务的自动化性质，可能与应对不确定、创造性工作所需的方式存在显著差异，尤其是在层级性较弱、分布更广且有利于创造力的环境中。基于以上讨论，本研究旨在探讨以下研究问题。

a. 工作流自动化如何影响开源软件开发中的创新速度?

b. 不同的工作流自动化机制——机械化与编排——如何差异化影响面向维护的增量创新速度和面向新开发的实质性创新速度?

针对这些研究问题，我们分析了2019-2020年间超过4,500个GitHub代码库和280,000个相关议题的纵向观测数据，在嵌入式分层多方法桥接设计（Sarker等人，2025年）中结合了机器学习与计量经济学。我们的计量经济学分析进一步采用了前瞻滚动进入匹配法，并辅以互补的估计策略：在议题层面的创新速度分析中使用双重差分法和工具变量估计，在项目-月度面板数据对下游结果的分析中采用矩阵补全反事实估计。

我们首次提供了关于工作流自动化对开源软件创新速度及其微妙机制产生显著影响的实证证据。首先，我们发现采用GitHub Actions显著加速了整体开源软件创新进程，将平均问题解决时间缩短了10.1%（相当于4.3天）。更值得关注的是，通过对GitHub Actions配置的长期观察，我们进一步发现机械化与编排化这两种机制对维护导向型创新和开发导向型创新分别产生了不同的影响。

对于涉及错误修复和持续补丁等常规任务的维护型创新，机械化使解决时间减少了7.8%（相当于3.0天）。这表明，对可重复、自包含且定义明确的工作进行自动化，能有效提升维护型创新的速度。然而，对于以开发新功能等新颖复杂开发工作为特征的开发型创新，机械化的影响有限且不显著。相反，协调机制使开发型创新的解决时间大幅减少了16.6%（相当于9.1天）。这一结果表明，在开源软件开发中加速新颖复杂的创新需要协调机制，包括步骤之间、开发者之间以及阶段之间的自动化协调与沟通。此外，我们对下游效应的分析表明，工作流程自动化对社区贡献量产生积极影响，体现为已关闭问题数量增加了18.2%。它还提升了项目创新质量，伴随而来的是星标数增加16.5%、复刻数增加9.9%以及版本发布数增加15.6%。

本研究对相关文献做出了若干显著贡献。首先，我们通过将研究重点从内容自动化（如GitHub Copilot的影响）拓展至工作流自动化，丰富了新兴的软件开发自动化文献体系。我们的研究响应了Jain等人（2021）的倡议，并通过探索软件开发中新兴的流程自动化形式，延伸了Anthony等人（2023）与Adjerid等人（2023）的研究。与先前主要依赖自动化层级框架的研究不同，本研究区分了机械化与编排作为工作流自动化的两种关键机制，并检验了二者对开源软件开发中渐进式创新与实质性创新的差异化影响。我们的发现凸显了工作流自动化策略与具体流程需求相匹配的重要性，特别强调了在协作密集、创新驱动的环境中编排机制的关键作用。

其次，针对创新文献，特别是关于信息系统与数字创新的研究（Swanson 1994, Fichman et al. 2014），我们提供了关于新兴工作流自动化系统技术如何加速渐进式与实质性创新速度的新颖实证证据。尽管基础研究已确立信息技术（IT）及组织层面各类赋能因素的重要性（Davenport 1993），且近期研究强调了在线协作环境中协调对创新的关键作用（Sherif et al. 2006, Nambisan et al. 2017），但这些新兴工作流自动化技术对数字创新的具体影响与机制当前研究仍显不足。通过概念化并实证区分两种工作流自动化机制——机械化与编排化，我们发现：机械化能加速维护导向的创新速度，而编排化在加速开发导向的创新方面尤为有效。

第三，对于开源软件开发文献而言，我们的研究首次揭示了工作流自动化技术如何影响开源软件的创新速度与质量。既往研究主要强调开源开发独特的协作性及通常松散耦合的创新环境（Howison与Crowston 2014，Lindberg等 2016），并聚焦于传统的人力协作与治理模式（Shah 2004，Singh等 2011）。本研究通过深化对工作流自动化在开源开发中作用的理解，对现有文献形成重要补充。在管理实践层面，这项研究也为开源贡献者、维护者及平台方设计工作流自动化策略与工具包提供了可操作的指导。尽管我们的实证场景设定在开源领域，但关于不同自动化机制如何加速各类创新的洞见，同样适用于其他创新密集型场景——例如初创企业或敏捷团队——这些场景普遍具有高度创造性、紧密协作性及灵活工作结构的特点。

2. 相关文献

2.1. 自动化与开源软件开发

自动化改变了工作组织方式(Acemoglu and Restrepo 2019)。在企业内部，流程自动化——即运用技术重构、增强和加速业务流程与职能——已在多个领域得到研究，包括办公系统(Millman and Hartwick 1987)、自动化销售代理(Adam et al. 2023)、医疗保健领域的算法驱动流程创新(Adjerid et al. 2023)，以及电子商务中的工作流管理(Georgakopoulos et al. 1995, Basu and Kumar 2002)。从历史维度看，这类自动化多可归类为机械化——即在执行独立、常规任务时用技术替代人力劳动。这种自动化形式主导了整个二十世纪制造业及其他领域的讨论(Gunn 1982, Smith 1982)。随着工作日益复杂且相互关联，从业者开始将自动化不仅视为机械化，更视作流程编排。在信息安全与网络技术等领域，流程编排——协调管理跨流程及利益相关方的信息流与相互依存关系——已日益重要；机械化实现的是单个任务自动化，而流程编排则构建工作流体系并实现信息交换自动化。

在流程自动化技术发展的同时，近年来随着人工智能进步推动的内容自动化也日益受到关注。生成式人工智能工具已展现出提升数字艺术（Zhou and Lee 2024）和写作（Doshi and Hauser 2024）等领域创意生产力与输出质量的潜力，尽管实际成效取决于人机交互方式及用户技能水平（Chen and Chan 2024）。在软件开发领域，越来越多的研究正在探索AI辅助编程工具对开发者生产力与代码质量的影响（Cui et al. 2024, Song et al. 2024, Yeverechyahu et al. 2024）。我们的研究从不同视角切入自动化议题，既不聚焦内容自动化，也不探讨自动化层级框架（Parasuraman et al. 2000, Kaber 2018），而是专注于流程自动化——具体而言，是研究流程自动化系统如何通过机械化与流程编排这两种概念机制影响开源软件开发。

开源软件（OSS）环境为考察流程自动化对协同创新的影响提供了特别宝贵的研究场景。OSS项目高度依赖分布式、自愿参与的协作模式，其正式权威结构有限，这使得协调与工作流程设计显得尤为重要（Mockus等人，2002；Howison与Crowston，2014）。先前的信息系统研究已广泛记录了各类因素如何影响OSS的发展路径与成果，包括贡献者角色（Setia等人，2012；Safadi等人，2021）、社交网络结构（Fershtman与Gandal，2011；Singh等人，2011）以及协调机制（Lindberg等人，2016）。近年来，研究进一步探讨了组织参与度与结构特征如何影响OSS开发。Medappa与Srivastava（2019）的研究表明，任务叠加程度等工作结构特征会显著影响项目成功率，而组织所有权则调节这些效应。类似地，Chen等人（2022）发现技术收购会重塑组织内外部对OSS的贡献，其影响程度取决于收购方的OSS经验与项目相似性。这些研究共同强调，OSS环境对协调挑战具有高度敏感性。基于此研究脉络，我们进一步探究流程自动化如何影响OSS中的创新速度，重点关注两种机制在两类创新中的差异化效应。

2.2. 创新与开源软件开发

信息系统研究早已确立信息技术作为创新的关键引擎，通过构建信息系统创新的类型学，并承认信息技术既是流程创新也是产品创新的源泉（Swanson 1994）。当前该领域正被数字创新——即"由信息技术赋能的产品、流程和商业模式"所根本重塑，这些创新需要对组织架构、组织、个人及其他利益相关者(Fichman et al. 2014, p.2)。随着软件成为跨行业创新的支柱，加速软件开发的机制已成为具有重大经济意义的课题(Branstetter et al. 2019)。

数字平台与在线社区中创新的本质与动态不断演变，其中社会技术因素塑造着协作成果（Ramaraju等人，2025年）。通过互助行为进行的知识交流影响个体产生创新想法的能力（Hwang等人，2019年），而沟通可见性则提升集体的“元认知”，从而催生更多创新并减少重复性工作（Leonardi，2014年）。传播速度与可再处理性等沟通能力能加强同行联系、促进共享知识库的访问，从而维系持续创新（Liao等人，2024年），尽管多元连接对于整合知识与管理协调挑战仍然至关重要（Kane与Ransbotham，2016年）。

OSS开发是这种协同创新模式的典型代表，其去中心化治理和自愿贡献机制为创新营造了有利环境。然而，这些特性也带来了显著的协调挑战——包括开发依赖（例如需要跨模块同步修改的功能）与开发者依赖（例如整合不同专业知识的专家）——这些都可能阻碍创新速度（Lindberg等，2016）。我们通过概念性提出并实证分析工作流自动化系统如何通过两种不同机制加速OSS创新，拓展了OSS研究领域：一是机械化机制——自动化离散的常规任务，二是协同编排机制——协调信息流与任务依赖关系。我们进一步探讨了这两种机制对维护导向的渐进式创新与开发导向的实质性创新产生的差异化影响。

3. 假设

本节为开源软件开发背景下的创新速度、以维护为导向的增量创新本质以及以新开发为导向的实质性创新提供了概念背景。我们同时讨论了工作流自动化及其以机械化与编排为形式的关联机制。基于以上讨论，我们提出三个假设：工作流自动化对开源软件创新速度的总体影响（假设1），以及不同自动化机制对两类创新产生的差异化影响（假设2a与假设2b）。

3.1. 工作流自动化与开源软件创新速度

尽管大量创新文献都强调创新数量——即新事物的数量

随着时间推移推出的产品和功能——速度是创新绩效同等关键的维度。创新速度指创意转化为已部署产品或功能的速度(Kessler and Chakrabarti 1996, Markman et al.2005)。在软件开发领域，这对应着IT行业时钟速度的概念，快速的技术变革和压缩的产品周期要求加速创新进程(Mendelson and Pillai 1998, Carrillo 2005)。由于数字技术的动态特性及近乎零边际成本的部署特性，创新速度在软件开发中尤为重要。与实体产品不同，软件可持续更新，能快速响应用户反馈、修复漏洞并推出新功能。响应用户需求的延迟会导致满意度下降、信任流失并最终转向替代解决方案，而快速迭代则有助于维持用户参与度和长期竞争力。

开源软件（OSS）进一步呈现出一种独特的场域特征，其以众包化、去中心化、自愿参与为核心（Raymond 1999, AlMarzouq et al. 2005）。贡献者通常地理上分散，不存在雇佣关系，其驱动力来自经济利益之外的一系列多元动机，包括技能提升（Hann et al. 2013）、声誉建立（Wasko and Faraj 2005）以及意识形态承诺（Lerner and Tirole 2002）。这类环境具有正式管理和层级监督极少的特点，因而天然具备自主性、官僚主义减弱和松散耦合等特性，这些特性被广泛认为有利于创造力、学习以及新想法的诞生（Ambile 1988, Woodman et al. 1993, Perry-Smith and Shalley 2003）。

尽管开源软件创新过程前景广阔，但其往往需要大规模协作（Levine and Prietula 2014），涉及异质且分布广泛的贡献者（Setia et al. 2012, Safadi et al. 2021），这使得创新过程复杂化，并可能导致延误与失败。开源项目通常过度依赖重复且繁重的手动工作，例如测试、更新、构建管理与部署，这些环节形成了阻碍交接、迟滞进展的瓶颈。即便引入了自动化流水线，也常因贡献者间各自为政而缺乏统一整合，导致效率低下甚至引发冲突（例如"在我机器上运行正常"的问题）。此外，项目还面临更广泛的协调挑战，包括跨开发阶段及分布式贡献者之间的通知、更新、触发机制、问题分类、工作序列管理及信息流转。正是在这种繁重工作负荷与协调挑战的动态交织中工作流程自动化可能会产生显著的影响。

尽管初期可能存在对开销或设置工作量的担忧，但我们认为现代工作流自动化系统（如GHA）能很好地应对这一挑战。与传统系统相比，模块化且可复用的操作机制允许逐步适配，从而抵消了初始成本，而标准化的YAML（YAML Ain't Markup Language）配置使设置和调试过程显著简化。更重要的是，通过自动化重复性任务和优化协作流程所带来的潜在效率提升是巨大的。

在诸如开源软件（OSS）这样的众包环境中，创新通常由用户驱动，通过问题请求启动并以问题解决告终。因此，创新速度的提升体现在问题更快地得到解决。通过系统性地减少人工开销并缓解协调瓶颈，工作流自动化有望产生净积极影响，将这些效率转化为更快的问题解决。因此，我们提出以下观点。

假设1.
工作流自动化（即启用GHA与未启用相比）能加速开源软件开发中的整体创新速度（即问题解决速度）。

在提出工作流自动化对开源软件创新速度的整体影响后，我们现在转向探讨机械化与协同编排这两种不同机制如何差异化地加速特定类型的创新。维护与新开发代表了软件工程中两种截然不同的工作形式，这一区分在软件开发与信息系统文献中已有充分确立（Kriebei and Raviv 1980, Banker et al. 1991）。我们遵循这一观点，将两者均视为开源软件创新的形式，这与Gupta等人（2006）提出的观点一致。基于创新文献中既有的区分，我们将面向维护的创新概念化为与渐进式创新相一致，强调局部改进与系统稳定性的保持；而面向开发的创新则对应实质性创新，涉及功能上更为根本性的增强或扩展（Dewar and Dutton 1986, Henderson and Clark 1990, Garcia and Calantone 2002）。这一概念框架为研究机械化和协同编排如何塑造不同的开源软件创新路径提供了基础。

3.2. 机械化与渐进式创新

机械化是指利用自动化工具（例如GHA、Travis CI）在软件创新周期内执行离散任务。这些离散任务是

通常具有定义明确、自成一体且可重复的特性。机械化的目标通常是通过减少或替代常规开发操作中的人工输入来提升一致性、可靠性和效率，确保标准化执行。例如在GHA工作流自动化系统中，特定操作可自动执行代码提交后的测试套件运行、将源代码编译为可执行文件、对代码进行格式规范检查，以及在测试通过后自动将代码部署至预发布或生产环境。

增量式创新通过常规且可预测的改进来优化现有功能、产品和流程。这种创新通过利用现有资源来维持现状（Gupta等人，2006），确保软件的可靠性和稳定性，保持强大而可靠的功能，并通过既定实践和标准化实现效率。在软件开发中，增量式创新通常通过维护工作体现，主要包括与错误修复、补丁更新和代码重构相关的问题，这些任务通常遵循明确的解决路径。鉴于其创造性和战略不确定性较低，增量式创新的速度较少受智力挑战限制，更多受制于操作效率低下。主要瓶颈在于执行这些变更的验证和实施过程中，所涉及的常规、可机械化任务所需的累计时间、精力和人工负担。

解决这类运营瓶颈恰恰是机械化所擅长的领域。增量创新不要求大规模头脑风暴、复杂协调或创造性问题解决，而是涉及机器能够可靠执行的离散、自包含且可重复的任务，例如运行测试套件、执行代码风格检查或编译代码。通过专注于高效、一致地执行这类已知任务，机械化强调对可机械化操作的标准化、自动化与流程优化。此处的创新加速源于消除了执行瓶颈：若无机械化机制，开发人员的进度常因需要手动执行并监控这些必要却常规的任务而受阻。试想开发人员实施单行错误修复的场景：尽管这项工作对智力要求极低，但运行完整的回归测试套件对质量保障仍至关重要。此类测试代表着纯粹的执行开销——一种耗时且需要切换工作情境的活动，虽属常规可重复操作，却阻碍了进展。机械化通过将此类速率限制任务卸载至自动化系统，彻底消除了该瓶颈。尽管实施机械化可能带来适应成本，但我们预期效率提升将远超过渡期的投入。因此，我们提出以下观点。

假设2a。在开源软件开发中，机械化（即在GitHub Actions中配置与不配置机械化操作）会加速增量（以维护为导向的）创新。

3.3. 编排与实质性创新

编排超越了离散任务执行，其定义为运用自动化工具（如GHA、Airflow）来协调任务序列与贡献者、分类管理复杂依赖关系，并促进跨开发阶段及团队成员的信息流转。这一机制应对了创新过程中协作与协调的复杂性（Dhanaraj与Parkhe 2006），涵盖了对超越远距离协调机制（如开源软件社区为应对此类复杂性所建立的模块化机制）能力范围的相互依赖关系管理（Lindberg等 2016）。例如，在GHA工作流自动化系统中，编排操作包括：当议题或拉取请求被创建或更新时通知相关维护者；在潜在依赖更新或冲突引发问题前进行检测与标记；依据预定义规则和访问权限应用标签并分配任务；以及当早期步骤失败、测试中断或需要进一步行动时触发后续任务。

实质性创新涉及通过探索来追求新知识和新能力（Gupta等人，2006年）。在软件开发中，实质性创新通过新开发体现，通常包括创建新功能、实施架构转型以及尝试需要创造性协作解决问题的前沿方法。成功的实质性创新需要有效的协调机制来整合多元专业知识，以及促进创造性实验的自主性（Amobile，1988年；Woodman等人，1993年）。开源软件的松散耦合与开放特性为实质性创新创造了异常肥沃的土壤（Raymond，1999年；Perry-Smith与Shalley，2003年；Levine与Prietula，2014年）。开源项目吸引了从核心维护者到外围贡献者的多元化人才库，每个人都带来能激发变革性创新的独特视角与专业知识（Setia等人，2012年；Safadi等人，2021年）。然而，这一特性也带来了对新开发成功同等至关重要的重大协调挑战。与具有明确解决路径的维护任务不同，新开发经常遇到开发工作与开发者之间未解决的相互依赖关系（Lindberg等人，2016年），此时限制进度的主要瓶颈可能并非智力容量，而是协调复杂性——即协调协作流程、管理信息流。

并在松散连接的贡献者之间整合分布式知识。

解决协调瓶颈是编排展现其独特价值之处。与机械化仅自动化独立任务不同，编排通过自动化操作序列、管理依赖关系和促进多渠道通信，应对复杂协作流程的协调挑战。工作流编排实现了项目内部协调的自动化，通过精简新开发中的协作环节，为创造性贡献创造更优条件（Dhanaraj与Parkhe，2006）。这种确保信息在正确时间传达给正确人员的能力，对于涉及不同专业背景且互动模式尚不成熟的贡献者参与的项目尤为宝贵（Mockus等，2002；Perry-Smith与Shalley，2003）。当正式协调机制存在局限时（AlMarzouq等，2005；Levine与Prietula，2014），编排通过算法构建并管理对整合多元贡献至关重要的协作流程——其中某些贡献可能带来新颖的认知洞见，但需要及时获取他人反馈才能有效实施（Setia等，2012；Safadi等，2021）。

以GitHub平台上的一个例子来说明：一个关于引入新功能的议题，该功能将彻底改变项目的数据处理架构——这是一个可能具有突破性但在架构上具有颠覆性的创新。在没有统筹协调的情况下，这些实质性的创新努力从一开始就面临协调挑战：如何协调领域专家？他们的沟通应如何构建和追踪？如何追踪和更新与外部代码的依赖关系？统筹协调通过启动一系列行动，系统性地消除这些瓶颈：自动将议题归类为重大架构变更，通过适当的沟通渠道通知相关的核心维护者和领域专家，建立专门的项目看板来追踪讨论和实施进度，触发全面的依赖性和安全扫描，同时通知相关贡献者，并确保在整个创新生命周期中持续向利益相关者更新进展。

机械化通过自动化新开发中的某些离散任务（例如测试）来实现效率提升，而编排则发挥着更广泛的作用，它通过自动化协调、依赖关系和信息流，为驾驭创新周期中的实质性变革提供关键支撑。因此，我们提出以下观点。

假设2b.
编排（即在GHA中配置与不配置编排操作）能够加速OSS开发中的实质性（以新开发为导向的）创新。

4. 研究方法论

我们的整体研究设计将机器学习与计量经济分析相结合，遵循一种嵌入式分层多方法“桥梁”策略(Sarker et al. 2025)。为了将每个问题归类为维护或新开发，我们采用了一种机器学习方法，该方法结合了基于GPT-4o的大型语言模型分类与经过微调的领域特定模型(CodeBERT)，并辅以使用替代模型(Gemma-2-2B)和人工标注员的验证。经过验证的创新分类使我们能够在后续的计量经济分析中，分析工作流自动化机制对不同类型创新的差异化影响。

4.1. 研究背景

GHA（GitHub Actions）于2019年8月由GitHub正式推出，作为本研究的实证背景。GHA是一个强大而灵活的工作流自动化系统，旨在简化和优化软件开发生命周期中固有的多样化流程。它使开发者能够直接在GitHub仓库中创建自定义的自动化工作流，不仅超越了传统的持续集成与持续部署（CI/CD），更涵盖了更广泛的开发活动范围。这种与GitHub生态系统的深度集成，使GHA能够响应多种仓库事件，例如代码提交、拉取请求的创建或更新、问题评论以及版本发布。

当GitHub Actions首次推出时，它被广泛视为一项创新功能，因为它允许使用可复用的代码模块（即Actions）来自动化常见的开发任务。除了任务执行外，GitHub Actions还支持开发者通过协调一系列活动来编排工作流。开发者使用YAML语法定义这些定制化工作流，指定由特定事件触发的任务序列与步骤。这种细粒度工作流自动化能力支持系统化执行大量任务，确保一致性、减少人工开销，并实现复杂的协同操作。GitHub Marketplace极大增强了GitHub Actions的通用性——该市场托管着庞大且持续增长的开箱即用Actions库，这些Actions作为执行特定操作的可复用代码单元，覆盖范围从传统CI/CD任务（例如搭建特定编程语言环境、运行测试套件（如使用Jest或PyTest框架进行单元/集成/端到端测试）、编译代码、构建软件制品（如Docker镜像）以及将应用部署到各类托管平台（例如亚马逊云服务、Azure、Heroku））到更具协调性与沟通性的功能。例如，存在可根据问题与PR内容自动添加标签的Actions。

当特定工作流事件发生时（例如构建失败或使用类似slackapi/slack-github-action的操作成功部署），可通知内容或状态（例如操作/标签器）、向Slack或Microsoft Teams等通信平台发送通知、管理项目看板（例如alex-page/github-project-automation-plus）、从提交信息自动生成版本说明，或基于先前步骤的结果触发后续工作流，从而编排复杂的事件链。这套丰富的操作生态系统不仅使开发者能够自动化离散的机械任务，还能编排涉及信息传递、审批和多阶段流程的复杂序列。

GHA的若干关键特性使其成为研究自动化对软件开发创新微妙影响的特别合适场景，这与我们的研究重点高度契合：首先，通过YAML定义的工作流自动化各类任务，GHA极大减少了人工干预需求。这一特性直接支持对项目需求响应机制的研究。例如加速代码变更的测试与集成这类工作流，正是理解维护导向型创新如何受影响的核心所在。

其次，GHA允许创建根据项目需求定制的定制化工作流。这些工作流可以包含多个作业，每个作业又有多个步骤，按照精确的顺序执行命令并响应各种条件。这种定制复杂自动化序列的能力对于管理维护和新开发工作都至关重要。它为区分机械化和编排提供了实证基础，使我们能够研究替代自动化机制是否以及如何支持不同类型的开源软件创新——从维护的渐进性质到新开发的实质性需求。

4.2. 数据与测量

4.2.1.

数据来源与样本构建。为全面分析工作流自动化如何影响软件开发创新，我们构建了2019年1月至2020年12月期间的数据集。该时段在GHA于2019年8月全面上线后提供了充足的观察窗口，同时避免了其他自动化技术的干扰，例如2021年6月发布的GitHub Copilot(Gershgorn 2021)。

我们的数据整合了GH Archive的全面GitHub事件历史与GitHub应用程序编程接口（API）。我们通过多阶段流程构建样本，旨在确保处理组与对照组之间的可比性，同时解决选择偏差问题。首先，我们筛选出在2019年之前启动的活跃开发项目首先，我们筛选出在观察窗口期内展现出持续开发活动并持续活跃至2023年的项目。其次，我们采用前瞻性策略以减轻选择偏差（Bapna等人，2018年）。通过查询GitHub的GraphQL接口，我们检查"github/workflows"目录的提交日期以精确获取GitHub Actions的启用时间。处理组包含在2019-2020年间启用GitHub Actions的代码库，对照组则由同期未启用但在2024年6月前最终启用的代码库组成。该方法确保所有代码库均表现出启用GitHub Actions的内在倾向性，使我们能聚焦于启用时机的差异而非自动化倾向的根本区别。第三，我们在交错启用场景中采用滚动进入匹配法以提升处理组与对照组的可比性。该方法通过控制反映时变因素的两个维度——接受处理的倾向性与处理的相对时机（Witman等人，2019年），解决了传统倾向得分匹配法在交错进入场景中的局限性（Stuart，2010年）。与传统倾向得分匹配法在单一日历时间点进行匹配不同，滚动进入匹配法基于相同相对预处理时间评估的可比协变量来对齐处理组与对照组项目，确保特征与时机的双重可比性。在线附录C详细比较了传统倾向得分匹配法与我们实施的前瞻性滚动进入匹配法，并提供了基于t检验和标准化均值差检验的平衡性检验。

4.2.2.

动态工作流信息提取与动作分类。为追踪工作流自动化的启用与演进过程，我们实施了一套精细的数据提取流程。具体而言，我们通过GitHub GraphQL API遍历样本中每个仓库与工作流配置修改相关的所有提交记录（即对"github/workflows"目录的更改）。针对每个提交点，我们还利用GitHub REST API提取完整的YAML配置文件，以精准捕获自动化配置详情。这些YAML文件经过系统化解析，用于识别正在使用的动作，并依据下文将阐述的机械化与编排化分类框架进行归类。为便于实证分析，提交层级的工作流数据会映射至每个议题的时间脉络，从而刻画其自动化状态（例如：无自动化、机械化、编排化、或机械化与编排化并存）。该综合数据收集与处理流程的更多细节详见在线附录E。

为了从实证角度评估我们对机械化与编排的理论区分，特别是它们如何对维护导向和开发导向的创新产生不同影响，我们系统地

我们对超过23,600个GitHub Actions进行了分类。这一重要的第一步涉及收集GitHub Marketplace上所有可用Actions的元数据。针对每个Action，我们收集了市场平台定义的功能类别标签（例如持续集成、测试、项目管理、依赖管理）。随后，我们将这些市场类别标签映射到我们的双重理论分类框架中。这一映射过程基于对每个标签主要功能目的的深入分析：判断其主要功能是自动化离散的机械任务（机械化），还是促进开发过程中更广泛的信息流、协调与协作（流程编排）。对于同时具有两类特征的双标签Actions，我们将其归类为同时属于两个类别，以准确捕捉其混合特性。这种基于理论的分类方法确保了我们自动化机制的实证测量与概念框架保持一致。代表性类别标签及示例Actions见表1。所有GitHub Marketplace类别标签的详细分类及映射过程的理论依据详见在线附录D。

4.2.3. 创新类型分类（维护型与新开发）。为了从经验上区分我们数据集中问题的渐进式创新（维护）与实质性创新（新开发），我们开发并执行了一个多阶段分类方法，该方法结合了基于大语言模型（LLM）的分类与经过微调的领域特定模型。初始阶段涉及使用OpenAI的GPT-4o模型对随机抽取的10,000个GitHub问题（来自应用LA-REM前、包含70万+问题的完整数据集，以确保多样性）进行分类。我们选择该模型是因为其先进的自然语言理解能力，能够解读软件开发问题中细致入微的内容。遵循Yeverechyahu等人（2024）的做法，我们提示GPT-4o模型将每个问题归类为以下五个操作类别之一：（1）代码开发（例如，功能添加、新能力、根本性改进），（2）维护（例如，错误修复、重构、依赖项更新），（3）文档与风格（例如，创建或更新文档、代码样式变更）、(4)测试与质量保证（例如添加或更新测试、版本升级），以及(5)其他（如许可证变更）。我们将这五个操作类别系统性地对应到软件开发中维护导向与开发导向创新的理论构念。具体而言，涉及维护、文档与样式、测试与质量保证的问题——这些通常聚焦于对软件既有方面进行渐进式优化或验证——被合并归类为“维护类”。涉及代码开发的问题——通常包含创建新功能或对现有功能进行重大改动——则对应为“新开发类”。属于其他类别的问题被归类为“非相关类”，代表那些既不主要侧重维护也不侧重新开发的问题。

为将我们的分类方法扩展至超过70万个问题，我们微调了微软的CodeBERT模型——这是一个在自然语言和源代码上预训练的双模态深度学习模型（Feng等人2020年）。利用1万个由GPT-4o标注的问题数据，该模型被适配至我们的分类任务，并应用于完整数据集。该方法实现了软件开发中创新类型的大规模识别。为评估稳健性，我们还应用了谷歌的Gemma-2-2B模型，并观察到高度一致的分类结果（Google DeepMind 2024年）。表2展示了分类结果及示例。在线附录F提供了关于提示设计、理论映射、微调过程及人工验证的进一步细节。

4.3. 数据集与因变量

在此，我们详细报告了主要的问题级别数据集以及补充的项目-月份级别数据集。表3中展示了两个数据集中关键变量的汇总统计信息（包括原始值及对数转换值）。

4.3.1. 议题层面的重复横截面数据集。

我们的主要分析研究了工作流自动化对开源软件创新速度的影响。基于我们对基于众包的开源软件平台中用户驱动创新的讨论（第3.1节），我们通过问题解决来捕捉开源软件创新中的创新速度。

表1. 工作流自动化机制分类

	Mechanization	Orchestration
Example action tags	code-quality, continuous-integration, testing, deployment, container-ci, code-review, mobile-ci	project-management, chat, api-management, dependency-management, open-source-management, support, deployment-protection-rules
Example actions	golangci/golangci-lint-action uraimo/run-on-arch-action actions/upload-artifact	rtCamp/action-slack-notify actions/labeler kentaro-m/auto-assign-action

表2. 开源软件中创新性质的分类

	Maintenance	New development
Issue types	Maintenance, documentation and style, testing and quality assurance	Code development
Example issues	- Fix memory leak causing crashes - Resolve database connection timeout errors - Refactor duplicate code in utils library - Fix CSS styling inconsistencies	- Create new GraphQL API architecture - Implement Docker containerization - Add support for PostgreSQL database - Implement OAuth authentication framework

$time，定义为问题提交至解决之间的持续时间。每个问题及其对应的解决时间都封装了一个单元开源软件创新的生命周期：从社区成员最初的功能请求或错误报告开始，经过社区讨论和解决方案开发，直至实施与解决。因此，问题解决时间的缩短可作为衡量开源软件创新加速的直接且可观测的指标。相应地，这个包含项目与时间固定效应的问题级别重复横截面数据集，代表了研究开源软件开发中创新速度最合适的分析层级。具体而言，我们构建了一个精细数据集，涵盖2019-2020年研究期间匹配样本代码库中所有被开启的问题。在该数据集中，每个问题通过项目p及其在该项目中的序列i进行唯一索引。对于每个问题，我们捕获其特征（例如标题、内容、标签）及其开启与关闭的时间戳。我们将每个问题分类为维护类或新开发类，具体流程详见第4.2.3节。

该数据集的另一个关键特征是对每个项目的自动化配置进行纵向追踪。由于工作流由一个或多个操作组成，对于启用GHA的项目，我们会在每个议题开启时识别工作流中的具体操作配置，并进一步将GHA配置类型分类，即：无自动化、机械化、编排化，或机械化与编排化并存。这使得我们能够观察GHA配置在项目内部的时间变化，从而进一步通过实证评估不同工作流自动化机制的有效性。

我们关注的核心构念——创新速度，通过问题解决时间（以小时计并取对数）来操作化。这一因变量捕捉了问题开启与解决之间的持续时间。为增强稳健性，我们构建了替代测量方法以处理未关闭问题，包括将未关闭问题的时长上限设为六个月，以及创建二元指标来反映问题是否在30天、60天或90天内解决。这些稳健性检验结果详见在线附录G.4。在全面问题

表3. 描述性统计

	Observations	Mean	Standard deviation	Minimum	Maximum
Project-issue level data set					
Issue resolution time (hours)	280,282	1,014.6	2,132.6	0.0	18,189.8
Issue resolution time (hours, log)	280,282	4.6	2.6	0.0	9.8
Category (issue type)	Observations	Percentage			
Maintenance	208,573	74.4%	–	–	–
New development	71,709	25.6%	–	–	–
Project-month level data set					
Issues closed	109,152	3.8	15.9	0.0	3,317.0
Stars	109,152	15.9	59.1	0.0	6,472.0
Forks	109,152	3.4	13.4	0.0	727.0
Releases	109,152	1.0	4.8	0.0	256.0
Issues closed (log)	109,152	0.8	1.0	0.0	8.1
Stars (log)	109,152	1.5	1.4	0.0	8.8
Forks (log)	109,152	0.8	0.9	0.0	6.6
Releases (log)	109,152	0.4	0.6	0.0	5.5

注：由于四舍五入，汇总统计中解决时长可能显示为零；所有数值均为正数，由精确到秒级的时间戳计算得出，再转换为以小时为单位的时长，数据集中不存在真正的零时长解决情况。在在线附录G.4中，我们报告了敏感性测试，以排除可能被快速解决的琐碎问题。

结合LLM分类、微调与人工验证，我们的数据集涵盖了与维护或新开发相关的问题，这使我们能够研究不同的自动化机制如何差异化地影响这两种不同类型的开源软件创新。

4.3.2.

项目-月度层面面板数据集。我们首先从贡献量入手，在项目1月度层面汇总已解决问题的数量。我们同样纳入了项目-月度层面的创新成果衡量指标，这些指标反映了质量，例如仓库星标、复刻和新版本发布。星标体现了用户的认可度，复刻反映了开发者的参与度和协作潜力，而版本发布则代表了具体的创新里程碑。通过这些项目-月度层面的衡量指标，我们旨在全面分析贡献模式以及下游创新质量成果。

4.4. 实证策略

我们的实证识别策略根据我们两个数据集的不同特点进行了定制，遵循我们的LA-REM流程。对于包含项目和时间固定效应的议题层面重复横截面分析，我们采用双重差分法估计。对于项目-月份层面的面板数据分析，我们使用矩阵补全反事实估计。

4.4.1.

使用双重差分法进行议题层面分析。针对创新速度的分析，我们采用双重差分法作为主要的识别策略。我们利用议题创建时间与项目级GHA注册时间之间的时间变化来估计其影响。双重差分指标AfterGHA的定义如下：对于在其关联项目注册GHA之后开启的议题，AfterGHA等于1；对于在其关联项目注册GHA之前开启的议题，或其关联项目在观察窗口期内未注册GHA的议题，AfterGHA等于0。因此，我们的基线设定如方程(1)所示：

$$\log(\text{Resolution Time}_{pi}) = AfterGHA_{pi} \cdot \beta + \alpha_p + \gamma_{t(pi)} + \epsilon_{pi}. \quad (1)$$

此处，每个观测值由pi标识，其中p代表项目，i索引项目p内的每个议题。这类似于重复横截面数据结构。$t(pi)$ 表示项目p中议题i被提出的年月。指示变量 AfterGHA_{pi} 在项目p于议题i提出时间$t(pi)$已加入GHA时取值为1，否则为0；αp 代表项目固定效应，用于控制每个项目不随时间变化的特征，而γt(pi) 捕捉时间固定效应，以吸收在特定时间区间内对所有提出议题产生普遍影响的周期性冲击或趋势。双重差分指标 AfterGHA_{pi} 的系数β估计了平均处理对已处理者的效应（ATT），我们关注的估计目标。

4.4.2

基于矩阵补全法的项目-月度层面分析。在我们的项目-月度层面分析中，我们采用矩阵补全估计量来考察贡献量与项目质量。矩阵补全估计量属于一类专为面板数据设计的反事实估计方法，其处理时点在各单位间存在差异。该方法通过低秩矩阵近似未处理潜在结果的结构（Athey等人，2021），以应对处理效应异质性和时变混杂因素带来的挑战。这种方法特别适用于我们的研究场景——该场景具有交错的处理时点特征，且处理效应存在异质性，可能因项目而异并随时间演变。

该估计流程包含四个步骤：(a)仅使用未处理观测数据（项目加入GHA前的月份）拟合模型；(b)运用拟合模型为处理组观测数据（加入后的项目月份）估算反事实结果；(c)通过计算每个处理组项目月份的实际观测值与估算结果的差异得出个体处理效应；(d)汇总个体效应以估计平均处理效应。此方法的核心特征在于在观测层面构建反事实推断，从而允许处理效应在不同维度存在异质性（Liu等，2024）。MC方法采用正则化优化估计低秩矩阵L，在模型拟合度与简洁性之间取得平衡以避免过拟合。我们通过交叉验证选取合适的正则化参数，并采用项目层级的聚类块自助法进行统计推断。

5. 实证结果

本节报告实证研究结果，具体包括：(a)工作流自动化（即GHA注册）对开源软件创新速度（问题解决时间）的影响；(b)分别驱动不同类型软件创新（维护性开发与新建开发）速度提升的两种自动化机制（机械化与流程编排）；(c)社区贡献与创新质量的衍生影响；以及(d)一系列稳健性检验。

5.1. 开源软件创新速度

5.1.1.

平行趋势检验。我们首先报告了议题层面双重差分估计的平行趋势分析。鉴于我们的匹配样本包含超过4,500个项目，且每个项目每月平均仅开启三个议题，作为标准做法，我们以季度为单位估算相对于处理时间的相对时间单位。季度相对时间单位提供了

通过更稳定平滑的系数估计，有效降低了议题活动月度波动的干扰噪声，确保事件研究中每个项目在各时间段内拥有充足的议题数量。图1直观展示了各相对时间段的估计系数及其95%置信区间。通过可视化观察可发现，处理组与对照组项目的议题解决时间在GHA注册前呈现平行趋势，未显示出明显的预趋势特征。相关回归系数及正式预趋势检验结果详见在线附录G.1。

5.1.2.

主要双重差分法结果。表4报告了我们考察工作流自动化对创新速度影响的主要双重差分法结果。除了项目与时间固定效应外，列(2)进一步纳入了处理组别特定的线性时间趋势，以允许不同处理组在干预前后遵循差异化的发展轨迹。结果显示，工作流自动化(GHA)在两种设定下均显著加快了软件开发创新速度，且两列显示的估计值几乎完全一致。具体而言，列(2)中−0.107的系数表明，工作流自动化使问题解决时间缩短约10.1%(e−0.107−1)。鉴于样本中开源软件项目的平均解决时间为42.3天(1,014.6小时)，这意味着每个问题的处理时间减少约4.3天(102.5小时)。该效应在我们的基准双重差分设定(列(1))中保持稳健，效应量的差异可忽略不计。这种跨设定的一致性模式为工作流自动化提升开源软件开发创新速度提供了有力证据，支持了假设1。

表4. 工作流自动化对创新速度的影响

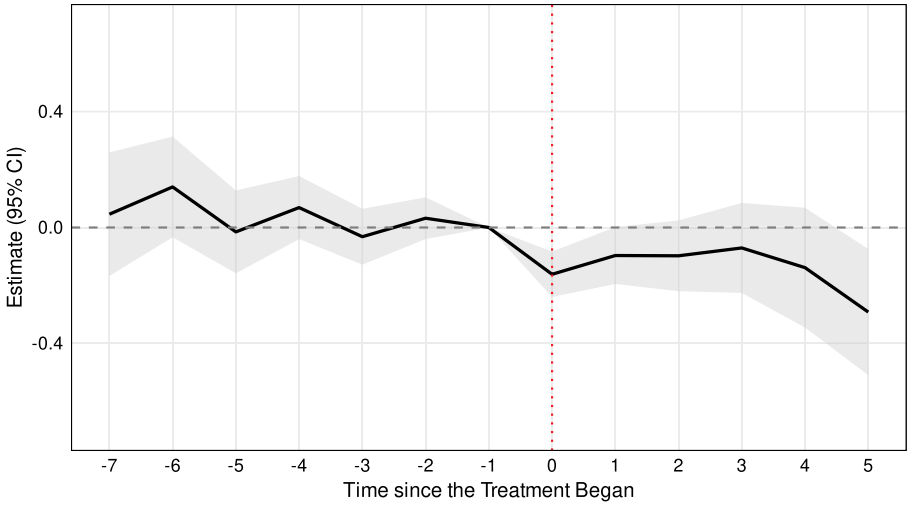
	Issue resolution time (h, log)	
	(1)	(2)
AfterGHA (β)	−0.106***	−0.107***
	(0.041)	(0.041)
No. of observations	280,282	280,282
R^2	0.177	0.178
Project fixed effects	Yes	Yes
Time fixed effects	Yes	Yes
Treatment cohort linear time trend	No	Yes

注：括号内报告的是稳健标准误，按项目层面进行聚类。

$S****<0.01.$S

问题解决平均提速4.3天，意味着整个开源生态系统取得了显著进步。虽然大多数开源贡献属于自愿行为，但参照市场劳动力成本计算，这种时间节省将转化为实际经济价值。基于150万个活跃项目及30%的GHA采用率，我们估算GHA每月能为GitHub平台节省约2.542亿美元劳动力成本（详见在线附录H）。除成本节约外，更快的解决速度通过降低贡献者流失率、维持项目发展势头、加速迭代周期来提升项目可持续性——这些因素在志愿者驱动的社区中尤为重要。在快速演进的技术环境中，创新速度往往决定竞争优势。对于关键任务型开源项目而言，4.3天的提速差异可能意味着能否及时发布关键安全补丁，或是用户流失至替代解决方案的分水岭。

图1.（彩色在线）平行趋势与动态处理效应



注：时期1为参照期。处理自时期0开始。

5.1.3

工具变量法。为补充双重差分法以估计工作流自动化对创新速度的平均效应，我们进行了工具变量分析，该分析提供了一种替代识别策略。该工具变量利用了项目接触GHA的差异性，这种差异性由其核心开发团队的外部活动所驱动。具体而言，我们在项目–月度层面构建了一个工具变量，该变量捕捉了截至每个特定月份，项目核心团队成员在GitHub上参与已启用GHA的外部项目的程度。我们根据每位贡献者对焦点项目的重要性（即以其贡献度作为代理）对此接触度量进行加权。这一加权接触变量背后的核心逻辑是，利益和影响力更大的贡献者更有可能成功倡导并推动项目启用GHA。此度量旨在反映核心开发者从其外部项目中带来的、不断演变的认知，从而为GHA的启用提供了一个差异性来源，该差异性与焦点项目开发轨迹中未观测到的冲击具有合理的独立性。关于此工具变量构建的详细说明，请参阅在线附录B。

我们的工具变量策略的有效性依赖于两个关键条件。首先，相关性条件要求工具变量与项目加入GHA的可能性高度相关。这一条件在设计上即得到满足：随着核心贡献者在外部项目中接触并积累GHA经验，他们更可能将GHA引入其主导项目，积极推动并促成其采用。其次，排他性约束要求工具变量仅通过影响GHA的采用来影响创新速度。通过利用贡献者在外部项目活动的变异，我们最大程度降低了工具变量捕捉到可能干扰估计结果的未观测项目特定冲击的风险。总体而言，工具变量方法为工作流自动化对创新速度的影响提供了另一种识别路径。

我们在表5中报告了工具变量回归的结果。我们的分析遵循与主双重差分法相同的设定，包括项目和时间的固定效应。内生变量AfterGHA（工作流程自动化注册）由核心成员对启用GHA的外部项目的加权暴露度作为工具变量。第一阶段估计表明，工具变量高度相关，其F统计量远超常规阈值，表明对GHA注册具有强预测力，从而缓解了弱工具变量的担忧。在第二阶段，工具变量AfterGHA的系数为负且统计显著，这意味着通过我们的工具变量渠道，对创新速度提升存在显著的局部平均处理效应（LATE）。

表5. 工作流程自动化对创新速度的影响（工具变量法）

	AfterGHA First-stage	Issue resolution time (h, log) Second-stage
IV: GHA Exposure	0.011*** (0.002)	
$\widehat{AfterGHA}$ *(Instrumented)*		−1.123*** (0.205)
First-stage *F*-statistic	6,429.2	
Observations	280,282	280,282
Project fixed effects	Yes	Yes
Time fixed effects	Yes	Yes

注：括号内报告了在项目层面聚类的稳健标准误。内生变量：AfterGHA。工具变量：核心成员在外部启用GHA的项目中的加权参与度。$*** p<0.01$。

IV方法为开源软件开发中工作流自动化对创新速度的影响提供了补充性识别与进一步证据。

5.2. 机械化与编排化

为进一步揭示我们主要发现背后的细微差别与作用机制，我们研究了不同类型的通用混合人工智能配置如何影响渐进式创新与实质性创新的速度。我们的理论框架预测：机械化配置对维护导向型创新最为有效，这类创新涉及定义明确、自包含且可重复的任务；而协调化配置应特别有利于开发导向型创新，这类创新需要跨多元主体的协同及隐性知识的整合。

如前所述，我们的数据使我们能够观察到GHA系统在交错式GHA注册之外是如何实施的。具体而言，我们观察了每个项目中GHA配置随时间的演变，不同问题在不同自动化配置下创建。例如，我们可能观察到，在项目的第10号问题创建时，没有GHA配置（即没有动作）；而对于第15号问题，GHA配置可能仅包含与机械化相关的动作；对于第20号问题，GHA配置可能由与机械化和编排都相关的动作组成。在实践中，项目很少单独实施编排；它通常建立在机械化的基础上，以协调更复杂的交互。与这一观察一致，我们排除了仅配置了编排动作的项目中的问题，这些仅占总数的0.09%。

我们采用相同期号的重复横截面数据集，考察全球健康援助的不同机制如何影响两类问题的解决时长。为引导分析，我们构建了一个分类

变量，自动化类型，包含三个层级：无自动化（参照组）、仅机械化、以及机械化结合编排。随后我们按问题类型对样本进行划分，区分维护类问题与新开发类问题。针对每种问题类型，我们采用如下模型设定进行估计：

对数（右侧 解决时间 p i ）

$$= \beta_1 \cdot \text{Mechanization}_{pi} + \beta_2 \cdot \text{Mechanization\&Orchestration}_{pi} + \alpha_p + \gamma_{t(pi)} + \epsilon_{pi}. \quad (2)$$

在方程（2）中，大部分符号与方程（1）相同，不同之处在于我们将 β1 解释为机械化的影响，将 β2 解释为机械化与编排的共同影响。差值 β2−β1 代表了编排的效果。值得注意的是，尽管主要分析中的处理变量（GHA 注册）是在项目层面发生的，但在此分析中，根据设计（GHA 配置），它是在议题层面运作的。遵循 Abadie 等人（2023）以及 Angrist 和 Pischke（2009）的观点——他们将标准误视为一个设计问题，并应在处理变量的层面构建——我们在议题层面报告了稳健标准误。我们还在在线附录 G 中为所有分析报告了采用替代标准误聚类的结果。

表6展示了采用我们先前设定的项目与时间固定效应模型进行机制分析的结果。对于维护导向的增量创新，机械化使问题解决速度提升了7.8%（e^(−0.081)−1）。鉴于维护导向创新的平均解决时间为37.9天（908.8小时），这意味着每个问题可节省约3.0天（71小时）。该结果为假设$2a$提供了实证支持，证实了在可预测且标准化的维护工作场景中，机械化能带来显著效益——自动化可有效精简单元测试等重复性任务。当我们进一步观察

表6. 不同工作流自动化机制与创新速度

	Issue resolution time (h, log)	
	Maintenance	New development
Mechanization (β_1)	−0.081***	−0.037
	(0.023)	(0.039)
Mechanization + Orchestration (β_2)	−0.133***	−0.218***
	(0.028)	(0.050)
No. of observations	208,079	71,480
R^2	0.174	0.261
Project fixed effects	Yes	Yes
Time fixed effects	Yes	Yes

注：括号内报告的是稳健标准误。无自动化组为参照组。

***$p < 0.01$.

在面向维护的创新中，我们发现这两种机制之间的影响差异并不显著（β2−β1=−0.052，t=1.44，p>0.1），这表明除了机械化本身能够实现的效果之外，协调机制对于面向维护的创新提供的额外影响有限。这一发现与我们的理论预期一致，因为对于自成体系且已有明确定义的任务，协调机制所带来的协作与知识共享益处并不那么关键。

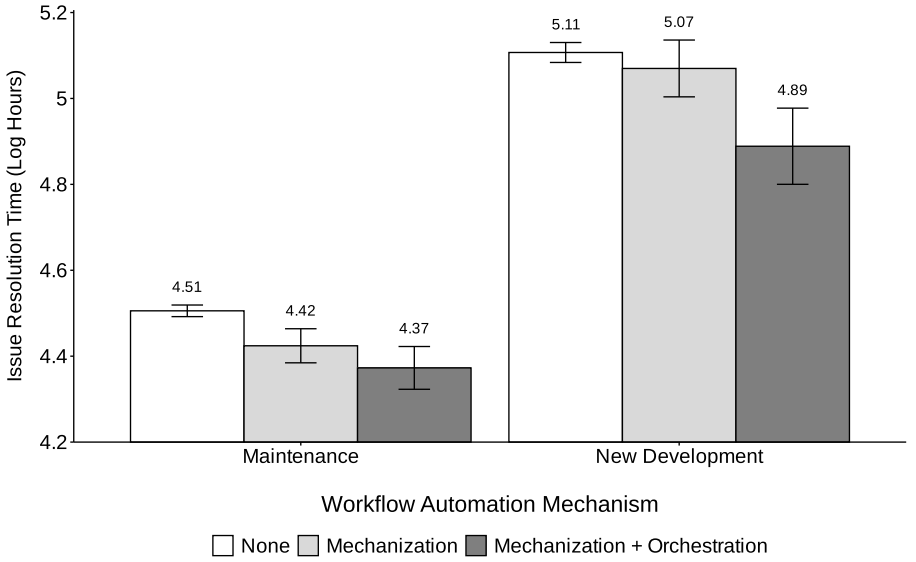
面向发展的实质性创新结果呈现出不同模式。机械化单独作用对开发问题未显示出显著影响（−3.6%，p>0.1），这表明仅靠机械化为需要新颖性和创造性的工作提供的价值有限。这支持了假设2b的前提：对于面向发展的创新，核心挑战不在于离散化、可机械化步骤的执行，而在于复杂相互依赖关系和知识流的管理——这些是单纯机械化无法解决的。然而当机械化与流程编排相结合时，效果显著增强：问题解决时间缩短19.6%（e^(−0.218)−1）（p<0.01）。这些系数间的差异具有统计显著性（β2−β1=−0.181，t=2.85，p<0.01），表明工作流编排是此类任务提速的关键驱动因素。我们进一步计算了编排机制在加速面向发展创新方面的经济效益，这相当于实现16.6%（e^(−0.181)−1）的创新速度提升，平均每个问题的解决时间缩短约9.1天（219.5小时）。

基于以上讨论，我们在此总结自动化机制对维护导向型与开发导向型创新的差异化影响。图2对比了三组采用不同工作流自动化配置的问题组，可视化展示了预测边际及其95%置信区间，并按照两类开源软件创新进一步分解了各项效应。柱体高度代表基于公式（2）计算的对数小时预测边际，其标准误差通过德尔塔法计算。虽然渐进式维护创新速度提升的主要驱动力是机械化，但实质性新开发创新则更多受益于流程编排。这一模式与我们的假设2a和假设2b相符：维护工作受益于消除重复性瓶颈的任务机械化，而新开发则需要促进沟通协调的工作流编排。

5.3. 下游结果

一个自然而然的问题是，观察到的创新速度提升是否以

图2. 按创新类型与自动化机制划分的问题解决时间



除了其他项目成果，如贡献量或项目质量外。为检验与创新速度提升相关的此类潜在权衡是否存在，我们采用矩阵补全方法，实证估计了GHA注册对一系列下游项目成果的影响，估计结果如表7所示。

研究结果显示，这对社区贡献量和项目整体质量产生了积极影响。从社区贡献来看，GHA注册显著提高了已解决问题数量18.2%，表明解决社区需求的能力得到增强。在项目质量成果方面，GHA注册使项目星标数增长约16.5%，表明社区认可度和赞赏度提升。同时，项目复刻数增长9.9%，意味着开发者对基于自动化项目进行构建的兴趣更为浓厚。最值得注意的是，项目版本发布量增长15.6%，表明开发周期有所缩短。

这些结果表明，工作流自动化在保证项目质量的同时，亚者加快了创新速度。采用GHA的项目能够处理更大规模的社区问题数量的增加并未成比例地增加开发者的工作量。工作流自动化系统的可扩展性在开源软件环境中尤为重要——在这些环境中，协调与沟通往往是限制项目发展的瓶颈。项目质量的提升（更多星标、分叉和版本发布）表明，这些运营改进能切实转化为项目的成功。能更快解决问题的项目更有能力保持发展势头、吸引用户和贡献者，并实现其发展目标。版本发布频率的提高意味着工作流自动化支持更宏大的开发周期，而星标与分叉数量的增长则反映出项目在开源社区内的可见度和认可度得到提升。

5.4. 稳健性检验

为了评估我们研究结果的稳健性，我们进行了一系列广泛的补充分析，这些分析总结在表8中，并在在线对应的附录中详细说明。除了最后两项，所有的稳健性检验都与创新速度分析相关，这些分析使用了重复的

表7. 工作流自动化对下游结果的影响（MC估计）

	Issues closed (log)	Stars (log)	Forks (log)	Releases (log)
AfterGHA	0.167***	0.153***	0.094***	0.145***
	(0.014)	(0.016)	(0.010)	(0.011)
Method	MC	MC	MC	MC
Treated observations	18,504	18,504	18,504	18,504
No. of observations	109,152	109,152	109,152	109,152
Project fixed effects	Yes	Yes	Yes	Yes
Time fixed effects	Yes	Yes	Yes	Yes

注：括号内报告的是稳健标准误，已在项目层面进行聚类处理。
***p<0.01。

表8. 稳健性检验汇总

Concern	Analysis	Details
DiD assumption	Event study plot, relative time model, pretrend tests	Section 5.1, Online Appendix G.1
Sample composition	Weighted versus unweighted specifications	Online Appendix A
Endogeneity	Instrumental variables with external GHA exposure	Online Appendix B, Section 5.1.3
Issue composition	Checks on issue compositional changes	Online Appendix G.2
Model specification	Alternative specifications (negative binomial, Cox hazards models)	Online Appendix G.3
Outcome measurement	Resolution time with capping, closure probability	Online Appendix G.4
Sample outliers	Excluding issues with quick resolution times and bots	Online Appendix G.5
DiD modeling	Alternative DiD estimators (CSDiD, 2SDiD, BJS estimators)	Online Appendix G.6
Spurious correlation	Permutation test with random reassignments	Online Appendix G.7
Sample sensitivity	Leave-out sensitivity tests (random and 10-fold)	Online Appendix G.8
Treatment granularity	Timestamp-based versus project-month level	Online Appendix G.9
Classification scheme	Gemma-2-2B issue categorization, human validation	Online Appendix F, Online Appendix G.10
Standard errors	Alternative standard error clustering at project-month level	Online Appendix G.11
MC assumption	Event study plot, pretrend tests	Online Appendix G.12
DiD modeling	Alternative DiD estimators for downstream outcomes (TWFE DiD, 2SDiD, CSDiD)	Online Appendix G.13

截面数据集用于问题层面的分析。最后两项检查涉及基于项目–月度层面构建的面板数据集进行下游结果分析。综合来看，这些稳健性检验证实了我们的研究结果在识别假设、估计方法和样本配置方面均具有稳健性。

6. 讨论

6.1. 主要发现

本研究从概念层面提出并实证评估了工作流自动化是否能够加速开源软件创新及其相关机制（机械化与编排）。通过整合机器学习与计量经济学的多方法研究，我们分析了超过4,500个GitHub仓库和280,000个议题的大规模数据集。运用双重差分法结合线性自回归误差修正模型，研究表明工作流自动化确实能促进开源软件创新，且这种影响具有复杂性——其效果随自动化机制的类型及开源软件创新的类别呈现差异化特征。

我们的首要关键发现是，工作流自动化（GHA）能显著加快整体创新速度，问题解决时间缩短10.1%，相当于每个问题平均节省约4.3天。其次，根据我们对机械化与流程编排的区分概念，我们通过实证研究证明了工作流自动化的这两种不同机制对渐进式创新和实质性创新产生差异化影响。具体而言，机械化主要有利于维护导向的创新，使相关问题解决时间平均减少7.8%（每个问题3.0天）。相比之下，对于开发导向的创新，机械化虽呈现负面但不显著的影响（−3.6%），表明其对新颖创造性工作的价值有限。

然而，编排机制对加速开发导向型创新展现出显著影响，使平均问题解决时间减少16.6%（即9.1天）。第三，创新速度的提升并未损害关键的项目级成果，如整体产出与质量。相反，工作流自动化的益处超越了创新速度的改进范畴。具体而言，我们的面板数据反事实估计表明，工作流自动化还能显著提升社区贡献能力：已关闭问题数量增长18.2%，这证明GitHub Actions（GHA）增强了项目高效处理社区请求的能力。此外，工作流自动化带来关键质量指标的显著改善：社区星标数增长16.5%，开发者复刻数增长9.9%，软件版本发布数增长15.6%。这些结果表明，工作流自动化产生了超越创新速度的实际效益，采用GHA的项目不仅获得更高参与度，还能产出更高质量的成果。

6.2 对研究与实践的启示

本研究对软件开发自动化文献做出了若干显著贡献。首先，我们提出了一个流程自动化视角，以补充软件工程领域新兴的内容自动化研究焦点（Song等人，2024；Yeverechyahu等人，2024）。不同于将自动化视为替代人类输入的单一或纯粹替代性结构（Raisch和Krakowski，2021；Gong和Png，2024），我们将开源软件中的工作流自动化概念化为由两个不同但互补的机制组成：机械化与协同编排。这种分解有助于阐明自动化如何超越单纯的任务执行来塑造创新动态。我们的研究也突破了层级化自动化的局限。

框架，例如从低到高自动化水平的连续体（Parasuraman等人，2000；Kaber，2018），通过强调机制之间的概念区分而非自动化程度。

其次，本研究通过考察新兴技术——工作流自动化——如何作为软件创新的新前因，对创新文献做出贡献。尽管先前研究已广泛确立信息技术在创新中的赋能作用（Swanson 1994; Fichman et al. 2014; Guo et al. 2023），并强调了影响数字创新成果的协调需求（Sherif et al. 2006; Nambisan et al. 2017; Hwang et al. 2019），我们特别关注不同工作流自动化机制如何影响开源项目中的创新速度。我们引入了一种新的类型学——机械化与编排化——以在概念上区分自动化机制，并通过将其映射到软件环境中作为以维护为中心和以开发为中心的创新（Dewar and Dutton 1986; Henderson and Clark 1990），从而具体化了渐进式创新与实质性创新之间的区别。我们的实证分析表明，机械化对应涉及离散且可重复任务的渐进式、以维护为导向的创新，而编排化则支持需要创造力和协调的更实质性、以开发为导向的创新。通过强调创新速度，我们的研究还揭示了创新的时间维度，这在软件开发研究中仍未得到充分探索（Kessler and Chakrabarti 1996）。尽管自动化常以运营效率为框架，我们的发现揭示了其更广泛的潜力：通过使组织在推进复杂开发工作的同时管理日常运营，从而扩展创新能力。

示了流程自动化技术如何能解决众包式分布式开发环境中创新潜力与协调挑战之间的根本矛盾。该领域文献已广泛论证了开放、去中心化的协作结构如何通过开放性、自主性和松散耦合等特征（Raymond 1999, Levine and Prietula 2014）为创新提供沃土。然而，这些激发创造力的特性同时也会引发显著的协调与沟通难题，进而可能制约创新潜力（Mockus et al. 2002, Howison and Crowston 2014, Lindberg et al. 2016），在开源项目涉及异质化贡献者时尤为突出（Setia et al. 2012, Safadi et al. 2021）。通过区分机械化与流程编排两种模式，我们阐明了不同自动化机制如何针对性地解决具体执行负荷与协调挑战。尤为重要的是，我们的研究结果表明，工作流编排有望减少松散耦合开发环境中的协调失败和沟通瓶颈，同时不会限制使开源软件有利于创新的灵活性和开放性。我们观察到的下游效应，包括更高的社区参与度和提升的项目质量，进一步证明工作流自动化可能形成一个良性循环，从而同时提升创新速度与质量。

实践层面而言，我们的研究结果为协调自动化策略与创新目标提供了有效指导。对于开源软件项目，追求稳定性和渐进式改进的维护者和开发者可从测试、格式化和部署等任务的机械化中获益。相反，当开源软件项目进入更具探索性或产品开发阶段时（例如构建新功能或重构架构），协调式自动化则更具价值，有助于优化协作流程。对于专注于快速创新的组织（如初创企业、研发团队或敏捷单元），我们的研究结果凸显了基于协调的自动化工具的战略意义。这些工具能在不确定的高强度开发周期中建立更紧密的反馈循环和更具适应性的协作机制，从而促进复杂创造性工作的开展。更广泛地看，机械化与协调化的分野不仅限于开源领域。虽然机械化对应常规操作性工作的自动化（如数据录入、文件处理、标准化计算），协调化则指向协作与管理职能的自动化（如审批流程、进度安排、路径规划、级联更新）。认识这种功能差异对于理解自动化对知识工作的深层影响至关重要，也为未来工作设计带来了重要启示。

6.3. 局限性与未来研究方向

我们承认本研究存在若干局限性，这些局限性也指明了未来研究的可行方向。首先，我们的发现基于对档案数据的计量经济学分析。鉴于研究设计的观察性质，我们无法操控GHA注册或配置，这限制了对研究结果的因果性解释。尽管我们采用互补的识别策略（双重差分法和工具变量法）来缓解部分常见的内生性问题，但仍无法完全排除潜在的未观测混杂因素，例如GHA注册决策中可能存在的自我选择问题以及稳定单位处理值假设的相关问题。未来研究在可行情况下可采用实验方法，以验证并扩展我们的发现。其次，我们的

实证研究基于开源软件开发的独特背景展开，因此我们提醒读者在未考虑情境细微差别的情况下，切勿将研究结果简单推广至其他场景。值得注意的是，开源软件开发具有众包式松散耦合的特性，且运作时组织层级极为精简。未来研究可探索本研究结论在其他组织环境中的适用性。第三，工作流自动化在改造软件开发以外的传统业务流程方面具有巨大潜力。随着Zapier和UiPath等企业为技术领域提供定制化高级自动化解决方案，未来研究可借助此类平台的数据，系统评估其广泛的组织影响。

在一个日益自动化的世界中，加速创新不仅需要任务执行的机械化，还需要协同工作流程的编排。本研究区分了这两种机制，并展示了它们对两类创新的不同影响。随着自动化系统承担更多常规操作任务，人类贡献的焦点可能转向那些受益于协同协调和复杂问题解决的活动。我们的研究结果表明，工作流自动化，特别是编排，在应对创新的操作性和协作性需求方面发挥着关键作用。值得注意的是，机械化与编排之间的界限具有历史偶然性：曾经需要编排的任务，随着时间的推移，可能变得可机械化。随着工作流自动化日益成为协同创作不可或缺的一部分，我们希望本研究通过将机械化和编排区分为工作流自动化的两种基本逻辑，为未来关于软件开发以及更广泛的数字媒介知识工作中自动化的研究提供概念基础。

尾注

GitHub Actions于2018年10月以有限公开测试版形式推出（Daigle 2018）。自2019年8月起，GitHub Actions开始支持CI/CD功能，并对所有公开项目免费开放（Friedman 2019）。

2 尽管与Travis CI和Circle
CI等成熟的CI/CD工具共享某些功能，GHA的特点在于其原生集成于GitHub平台内。此外，GHA不仅管理构建、测试和部署流水线，还能自动化更广泛的仓库事件和交互，例如直接管理问题、拉取请求、标签、分配、评论，并根据特定事件触发通知和反馈。

值得注意的是，我们的双重差分数据集类似于重复横截面数据结构，而非传统的单位-时间面板数据。我们在项目中观察到不同的问题及其对应的解决时间。因此，某些基于面板数据的方法（例如需要唯一项目-月度观测单位的反事实估计量）并不适用于此场景。然而，双重差分法能够很好地利用项目级GHA注册的渐进式变化来估计GHA的实际效果。

4 AfterGHA 等同于双重差分表达式 Treat ・After ，（π ），其中 Treat 表示项目 p 是否在我们的观察期内加入了 GHA，而 After ，（π ）是一个虚拟变量，当议题 π 在项目 p 加入 GHA 之后被提出时取值为 1。议题的提出时间由议题 π 唯一确定，我们将其表示为 t(π)。

5 我们感谢副编辑和一位匿名审稿人建议使用这一工具。

此处的工具变量策略识别出的局部平均处理效应，仅针对那些因其核心成员先前接触过该功能而启用GHA的项目子集。由于这些依从性项目可能反应更灵敏——其核心成员很可能已掌握有效实施自动化所需的知识——预计局部平均处理效应估计值将大于通过双重差分法或固定效应模型得到的平均处理效应或处理组平均处理效应。

参考文献

Abadie A, Athey S, Imbens GW, Wooldridge JM (2023)
何时应对聚类调整标准误？《经济学季刊》138(1):1-35.

Acemoglu D, Restrepo P (2019) 自动化与新任务：技术如何替代与恢复劳动力。J. Econom. Perspective 33(2):3-30.

Adam M, Roethke K, Benlian A (2023)
人类与自动化销售代理：客户反应如何及为何在销售阶段间转变。信息系统研究 34(3):1148-1168.

Adjerid I, Ayvaci MUS, Zzer O (2023) 算法驱动流程创新的价值：以脓毒症为例。Manufacturing Service Oper. Management 25(4):1545-1566.

AlMarzouq M, Zheng L, Rong G, Grover V(2005) 开源：概念、益处与挑战. 信息系统协会通讯 16(1):756-784.

曾发表于《组织行为研究》第10卷第1期，第123-167页。

Angrist JD, Pischke J-S (2009)
《基本无害的计量经济学：实证研究者指南》第一版（普林斯顿大学出版社，新泽西州普林斯顿）。

Anthony C, Bechky BA, Fayard A-L (2023)
《与人工智能"协作"：采用系统视角探索未来工作模式》。组织科学34(5):1672-1694。

Athey S, Bayati M, Doudchenko N, Imbens G, Khosravi K (2021)
针对因果面板数据模型的矩阵补全方法. 美国统计学会会刊 116(536):1716-1730.

Banker RD, Datar SM, Kemerer CF (1991)
评估影响软件维护项目生产率的变量模型。管理科学，37(1):1-18。

Bapna R, Ramaprasad J, Umyarov A (2018)
免费增值社区的货币化：付费升级是否提升了社会参与度？《管理信息系统季刊》42(3):719-736.

Basu A, Kumar A(2002) 研究评论：电子商务中的工作流管理问题。信息系统研究 13(1):1-14.

Branstetter LG, Drev M, Kwon N (2019)
《顺应程序：传统制造业中的软件驱动创新》。管理科学，65(2):541-558。

Brynjolfsson E (2023) 图灵陷阱：类人人工智能的承诺与危险。
载于《全球时代的增强教育》(Routledge, New York), 103-116页。

Carrillo JE (2005) 行业时钟速度与新产品开发节奏。生产与运营管理 14(2):125-141。

陈Z, 陈J(2024)
大语言模型在创造性工作中的角色：协作模式与用户专业能力的影响。管理科学.70(12):9101-9117.

陈伟、金芳、薛力（2022）繁荣还是消亡？技术收购对开源软件贡献的影响。《信息系统研究》33(3):867-886。

克劳斯顿 K (1997)《一种基于协调理论的组织流程设计方法》。《组织科学》第8卷第2期:157-175页。

崔志浩、德穆雷尔、贾菲、穆斯奥尔夫、彭斯、萨尔茨（2024）生成式人工智能对高技能工作的影响：基于三项软件开发者现场实验的证据。预印本，提交于9月5日，http://dx.doi.org/10.2139/ssrn.4945566。

Daigle K (2018) GitHub
Actions：由您构建，由我们运行。https://github.blog/enterprise-software/automation/action-demos/。

Davenport TH（1993）《流程创新：通过信息技术再造工作》（哈佛商学院出版社，波士顿）。

Dewar RD, Dutton JE (1986)
激进式与渐进式创新的采纳：一项实证分析。管理科学，32(11):1422-1433。

Dhanaraj C, Parkhe A（2006） 协调创新网络。《管理学会评论》31(3):659-669。

Doshi AR, Hauser OP（2024）生成式人工智能增强个体创造力但降低新颖内容的集体多样性。Sci.Adv. 10(28):eadn5290.

Feng Z, Guo D, Tang D, Duan N, Feng X, Gong M, Shou L, 等. (2020) CodeBERT: 一种面向编程语言与自然语言的预训练模型. Cohn T, He Y, Liu Y, 编.《计算语言学协会发现：EMNLP 2020》（计算语言学协会，宾夕法尼亚州斯特劳兹堡），1536-1547.

Fershtman C, Gandal N (2011)
直接与间接知识溢出：开源项目的“社交网络”。RAND J. Econom. 42(1):70-91.

Fichman RG, Dos Santos BL, Zheng ZE (2014)
数字创新作为信息系统课程中一个基础且强大的概念。MIS季刊 38(2):329-354.

Friedman N（2019）GitHub Actions 现已支持
CI/CD，公共仓库免费使用。https://github.blog/news-insights/product-news/github-actions-now-supports-ci-cd/。

Garcia R, Calantone R (2002) 对技术创新类型与创新性术语的批判性审视：一篇文献综述。J. Production Innovation Management 19(2):110-132.

Georgakopoulos D, Hornick M, Sheth A (1995)
工作流管理概述：从流程建模到工作流自动化基础设施。分布式并行数据库 3(2):119-153.

Gershgorn
D（2021）GitHub与OpenAI推出可自动生成代码的新人工智能工具，访问于2025年12月19日，https://www.theverge.com/2021/6/29/22555777/github-openai-ai-tool-autocomplete-code。

龚杰，Png IPL（2024）自动化促进专业化：实地证据。《管理科学》70(3):1580-1595。

Google DeepMind（2024）Gemma
2：在实用规模上改进开放语言模型。预印本，提交于7月31日，https://arxiv.org/abs/2408.00118。

Gunn TG (1982) 设计与制造的机械化。《科学美国人》247(3):114-130。

Guo F, Li Y, Maruping LM, Masli A (2023)
信息技术投资与信息技术人力资源的互补性：对不同类型企业创新的启示。信息系统研究 34(3):1259-1275.

Gupta AK, Smith KG, Shalley CE (2006) 探索与利用之间的相互作用。《管理学院学报》49(4):693-706。

Hammer M, Champy J (1993)
《企业再造：企业革命的宣言书》（Harpercollins出版社，纽约）。

Hann IH, Roberts JA, Slaughter SA (2013)
并非所有贡献都平等：开源软件社区中不同参与形式经济回报的考察。信息系统研究，24(3): 520-538。

Henderson RM, Clark KB (1990)
架构性创新：现有产品技术的重构与成熟企业的失败。Admin. Sci. Quart. 35(1):9-30.

Howison J, Crowston K(2014) 通过开放叠加的协作：开源之道理论。MIS季刊 38(1):29-50.

Hwang EH, Singh PV, Argote L (2019)
万事皆略通，数业有专攻：众包社区中的信息网络与创新。信息系统研究 30(2):389-410.

Jain H, Padmanabhan B, Pavlou PA, Raghu TS (2021)
人类、算法与增强型智能专题特刊：工作、组织与社会的未来。信息系统研究 32(3):675-687.

Kaber DB (2018) 人机自动化交互建模中的问题：自动化类型与层级框架的预设性方面。J. Cognitive Engrg. Decision Making 12(1):7-24.

Kane GC, Ransbotham S(2016)
研究笔记-内容与协作：在线同行生产社区中基于隶属网络的信息质量分析方法。信息系统研究，27(2):424-439。

Kessler EH与Chakrabarti
AK（1996）提出创新速度：一个关于情境、前因与结果的概念模型。《管理学会评论》21(4):1143-1191。

Kriebei CH, Raviv A(1980)
一种用于计算机系统生产力建模的经济学方法。Management Sci. 26(3):297-311.

Leonardi PM (2014)
社交媒体、知识共享与创新：迈向沟通可见性理论。信息系统研究 25(4):796-816.

Lerner J, Tirole J (2002) 开源软件的一些简单经济学原理。J. Industry Econom. 50(2):197-234.

Levine SS, Prietula MJ (2014) 开放式协作创新：原则与绩效。组织科学，25(5):1414-1433。

Li B, Huang N, Shi W (2025)
被迫改变？媒体对劳工问题的曝光与企业人工智能投资。信息系统研究，在线预印版，5月27日，https://doi.org/10.1287/isre.2022.0402。

廖冠宇、黄天立、丹尼斯·A·R、邓志伟(2024)
媒体能力对在线社区知识贡献的影响。信息系统研究 35(1):165-183.

林德伯格 A，贝伦特 N，加斯金 J，莱蒂宁
K（2016）在线社区中的相互依赖协调：一项开源软件项目研究。信息系统研究 27(4):751-772。

刘磊, 王洋, 徐阳 (2024)
时间序列截面数据因果推断中反事实估计的实用指南。美国政治学杂志 68(1):160-176.

Markman GD, Gianiodis PT, Phan PH, Balkin DB (2005)
创新速度：将大学技术推向市场。研究政策 34(7):1058-1075.

Maruping LM, Matook S(2020) 软件开发协调的演进：现状与未来研究议程. 欧洲信息系统杂志 29(5):443-457.

Medappa PK, Srivastava SC (2019)
叠加效应是否影响自由/开源软件项目的成功？对组织与个人开源软件开发的一项考察。信息系统研究，30(3): 764-786。

Mendelson H, Pillai RR (1998)
时钟速度与信息响应：来自信息技术产业的证据。信息系统研究 9(4):415-433。

Mikalef P, Benlian A, Conboy K, Tarafdar M (2025)
负责任的人工智能始于人工制品：挑战信息系统研究中负责任人工智能的概念。欧洲信息系统杂志 34(3):407-424。

Millman Z, Hartwick J(1987)
自动化办公系统对中层管理者及其工作的影响。MIS季刊，11(4):479-491.

Mockus A, Fielding RT, Herbsleb JD (2002) 开源软件开发的两个案例研究：Apache 与 Mozilla。《ACM 软件工程与方法学汇刊》11(3):309-346.

Nambisan S, Lyytinen K, Majchrzak A, Song M (2017)
数字创新管理：在数字世界中重塑创新管理研究。MIS季刊 41(1):223-238.

Osterloh M, Rota S(2007)《开源软件开发——集体发明的又一案例？》《研究政策》36(2):157-171.

Parasuraman R, Sheridan T, Wickens C (2000) 自动化交互中人类参与类型与层级的模型。IEEE 系统、人与控制论汇刊 A 辑：系统与人类 30(3):286-297.

Perry-Smith JE, Shalley CE (2003)
《创造力的社会维度：静态与动态社会网络视角》。Acad. Management Rev. 28(1):89-106.

Raisch S, Krakowski S（2021）人工智能与管理：自动化与增强化的悖论。Acad. Management Rev.46(1):192-210。

Ramaraju N, Pant S, Pant G (2025)
激发创新：发明者性别多样性对组合式创新的影响。信息系统研究，在线预印版，6月5日，https://doi.org/10.1287/isre.2023.0343。

Raymond E (1999) The cathedral and the bazaar. Knowledge Tech. Policy 12(3):23-49.

Safadi H, Johnson SL, Faraj
S（2021）谁贡献知识？在线创新社区中的核心-边缘张力。组织科学，32(3):752-775。

Sarker S, Bala H, Hong Y, Kankanhalli A, Rossi M, Gu B, Oestrei-cher-Singer G (2025)
推进信息系统领域的下一代多方法研究：一个框架及对作者与评审者的若干建议。信息系统研究。36(2):647-668.

Setia P, Rajagopalan B, Sambamurthy V, Calantone R (2012)
外围开发者如何为开源软件开发做出贡献。信息系统研究 23(1):144-163.

Shah SK(2004) 理解开放与封闭源代码软件开发社区中参与与协调的本质。Acad. Management Proc. 2004(1):B1-B5.

Sherif K, Zmud RW, Browne GJ(2006)
管理破坏性信息技术创新中的对等冲突：软件复用的案例。MIS季刊，30(2):339-356。

Singh, Tan,
Mookerjee（2011年）网络效应：结构性资本对开源项目成功的影响。《管理信息系统季刊》35卷4期，第813页。

Smith A (1982) 工作的机械化。《科学美国人》247(3):66-75。

Song F, Agarwal A, Wen
W（2024）生成式人工智能对协作开源软件开发的影响：来自GitHub Copilot的证据。预印本，提交于10月2日，https://arxiv.org/abs/2410.02091。

Stuart EA (2010) 因果推断的匹配方法：回顾与展望。统计科学，第25卷，第1期，第1-21页。

Swanson EB (1994) 组织间的信息系统创新。Management Sci. 40(9):1069-1092.

Wasko MM, Faraj S(2005)
我为何要分享？审视电子实践网络中的社会资本与知识贡献。MIS季刊 29(1):35-57.

Witman A, Beadles C, Liu Y, Larsen A, Kafali N, Gandhi S, Amico P 等人 (2019)
健康服务研究中存在滚动入组时的对照组选择：滚动入组匹配。《健康服务研究》54(2):492-501。

Woodman RW, Sawyer JE, Griffin RW (1993) 组织创造力理论初探.《管理学会评论》18(2):293-321.

Yeverechyahu D, Mayya R, Oestreicher-Singer G (2024)
大型语言模型对开源创新的影响：来自 GitHub Copilot
的证据。预印本，提交于9月12日，https://arxiv.org/abs/2409.08379。

Zhou E, Lee D (2024) 生成式人工智能、人类创造力与艺术。PNAS Nexus 3(3):pgae052。

《信息系统研究》的版权归INFORMS：运筹学与管理科学研究所所有，未经版权所有者明确书面许可，其内容不得复制或通过电子邮件发送至多个站点。此外，内容不得用于任何人工智能工具或机器学习技术。但用户可为个人使用打印、下载或通过电子邮件发送文章。